

ISM2411 — Lab Week 02

First Terminal Session & First Python Script — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 02 · Unit 1 · Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Segments	3
6	Segment-by-Segment Walkthrough	4
6.1	Intro (0:00–0:05)	4
6.2	Exercise 1 — Terminal Warm-Up (0:05–0:13, 8 min)	4
6.3	Exercise 2 — Verify Python (0:13–0:18, 5 min)	6
6.4	Exercise 3 — Write <code>hello.py</code> (0:18–0:26, 8 min)	7
6.5	Exercise 4 — Path Practice (0:26–0:32, 6 min)	8
6.6	Exercise 5 — Make a Sub-Project (0:32–0:40, 8 min)	10
6.7	Exercise 6 — Deliberate Error Experiment (0:40–0:50, 10 min)	11
6.8	Exercise 7 — Run from the Wrong Directory (0:50–0:58, 8 min)	13
7	Wrap-Up (last ~7 minutes)	15
8	Appendix A — Full Command & Code Reference	15
9	Appendix B — Extra Practice (only if the class finishes early)	16

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 02 Lab — First Terminal Session & First Python Script
Unit	Unit 1 · Foundations
Class length	75 minutes
Format	Live code-along, but in the terminal rather than an editor for most of the session — students type every command themselves, on their own machine, in real time
Prerequisites	Module 01 vocabulary (file, directory, path); Python and VS Code already installed per the precourse setup page
Student-facing lab page	markumreed.github.io/ism2411 — week02_lab
Exercises covered	Exercises 1–7 (required)
Submission	Screenshot of both <code>hello.py</code> outputs (Exercises 3 and 5) plus <code>python3 --version</code> output, to Canvas

This is the highest-variance lab of the semester to run, because it is the first time every student's *environment* — not just their code — is on the critical path. A perfectly typed command fails for a student whose `PATH` isn't set up correctly, or who installed Python without checking "Add to `PATH`," or who has both Python 2 and 3 aliased confusingly. Budget real slack in your pacing for environment troubleshooting; the exercises themselves are short, but "my terminal says command not found" will eat real minutes for 10–20% of a typical room. This guide's timing plan already assumes that friction.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Navigate the file system entirely from the terminal — `pwd`, `ls`, `mkdir`, `cd` — without touching a mouse.
2. Explain the difference between the current working directory and the filesystem overall, and predict what `pwd` will print after a `cd` command.
3. Write, save, and run a `.py` file from the terminal with `python3 filename.py`.
4. Read a Python traceback (`SyntaxError` or `NameError`) and extract the line number and error type from it, rather than treating it as unreadable noise.

5. Explain why running a script depends on your current directory, and correctly use a relative path (`python3 module02/hello.py`) to run a script from a different location.

3 Before Class — Setup Checklist

- ☐ Confirm your own terminal, editor, and Python installation work end-to-end before class — run every exercise below yourself first, on the platform you'll be demoing (Mac terminal commands differ slightly from Windows; know both, since your room will have both).
- ☐ If teaching a mixed Mac/Windows room, prepare the **Windows equivalents** of every Unix command up front (see the sidebar table in Exercise 1) — do not assume Windows students will silently translate `pwd` to `cd` (no args) on their own.
- ☐ Have the course's precourse setup page open in a browser tab, ready to link any student whose Python installation is broken — do not attempt to debug a from-scratch broken install live in front of the whole room; triage and route them to office hours or a TA instead, so the rest of the class doesn't stall.
- ☐ Decide your file-explorer policy for this lab: **no GUI file management today** — if a student opens Finder/Explorer to make a folder instead of using `mkdir`, redirect them back to the terminal. The entire point of the lab is muscle memory for terminal navigation.

4 Materials Needed

- Terminal (Terminal.app on Mac, PowerShell or Command Prompt on Windows, or VS Code's integrated terminal on either — recommend the VS Code integrated terminal for a consistent look across platforms if your room allows it)
- VS Code (or any text editor) for writing `.py` files
- Python 3.10+ and pip, already installed per precourse setup

5 Segments

This plan uses all 75 minutes across the seven required exercises. Exercises 6 and 7 carry this lab's most important ideas (reading errors, understanding why the current directory matters), so protect their time if the room runs slow.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

“Today you stop clicking icons to navigate your computer and start typing commands instead. This feels slower at first — it is not, once it’s muscle memory, and every tool you’ll use in this course and in most data-analysis jobs assumes you’re comfortable in a terminal. We’re also writing and running your very first real Python script today. Two things to keep an eye on for yourself: does `python3 --version` actually work on your machine, and does the terminal open in a sensible starting location? If either is broken, raise your hand now rather than fighting it silently for twenty minutes.”

Do: Quickly poll the room — “hands up if `python3 --version` already works for you, right now, before we start” — this takes 30 seconds and tells you immediately how much environment triage you’re likely to be doing during Exercise 2.

6.2 Exercise 1 — Terminal Warm-Up (0:05–0:13, 8 min)

Teaching goal: Four foundational commands (`pwd`, `ls`, `mkdir`, `cd`) that every later exercise depends on — students need these to be reflexive, not looked-up, by the end of this block.

Say to the class:

“Four commands. `pwd` tells you where you are. `ls` tells you what’s here. `mkdir` makes a new folder. `cd` moves you into one. That’s the whole toolkit for today’s navigation.”

Live-code this, one command at a time, narrating the output each time:

```
pwd
ls
mkdir ism2411
cd ism2411
pwd
```

Line-by-line explanation:

- `pwd` — “print working directory”: shows the full absolute path of where your terminal currently is. This is the terminal’s answer to “where am I,” and it’s the single most useful debugging command in this whole lab — when anything goes wrong later, `pwd` is the first thing to run.
- `ls` — lists the contents of the current directory (files and folders). On Windows PowerShell this is also `ls` (aliased) or natively `dir` — mention both so Windows students aren’t thrown by seeing `ls` work in a tutorial written with Mac in mind.

- `mkdir ism2411` — creates a new directory named `ism2411` *inside* the current directory. No output on success — this trips up beginners who expect confirmation; say explicitly “no news is good news” for this command, and follow it immediately with `ls` if you want visual proof.
- `cd ism2411` — “change directory”: moves your terminal’s current location *into* the `ism2411` folder you just created. Also produces no output on success.
- `pwd` (second time) — confirms the move worked; the printed path should now end in `/ism2411` (or `\ism2411` on Windows).

Platform command reference (keep this visible if teaching a mixed room):

Task	Mac / Linux	Windows (PowerShell)
Show current location	<code>pwd</code>	<code>pwd</code> (or <code>Get-Location</code>)
List contents	<code>ls</code>	<code>ls</code> (or <code>dir</code>)
Make a folder	<code>mkdir name</code>	<code>mkdir name</code>
Enter a folder	<code>cd name</code>	<code>cd name</code>
Go up one level	<code>cd ..</code>	<code>cd ..</code>

Run it. Expected output (final `pwd`):

`/Users/yourname/ism2411`

(or the Windows equivalent, e.g. `C:\Users\yourname\ism2411`)

Common student mistakes to watch for:

- Running `mkdir ism2411` a second time by accident (e.g. after already `cd`-ing into it) — this either errors (File exists) or, depending on OS, creates a *nested* `ism2411/ism2411/` folder. If you spot a student inside a doubly-nested folder, this is a good moment to have them run `pwd` and read the output out loud themselves to spot the problem, rather than you pointing it out first.
- Typing `CD` or `PWD` in all caps out of habit from other software — commands are case-sensitive on Mac/Linux terminals (Windows is more forgiving here, another good cross-platform note).
- Confusing `mkdir` (make a folder) with actually *entering* it — a student runs `mkdir ism2411` and then immediately tries the next exercise’s commands, forgetting they’re still in the parent folder, not inside `ism2411`. Watch for this specifically, since it cascades into confusion for the rest of the lab.

Check for understanding: “If I run `ls` right now, what should I see?” (Nothing, or nearly nothing — `ism2411` is empty right after creation. This distinguishes `ls`, which reports on the *current* directory, from something that would report on the parent.)

6.3 Exercise 2 — Verify Python (0:13–0:18, 5 min)

Teaching goal: Confirm every student’s Python installation actually works *before* they need it for real code — and this is the highest-risk moment in the lab for environment problems surfacing.

Say to the class:

“Two commands. If either one fails or shows a version number lower than 3.10, don’t try to fix it alone during class — flag it to me right now.”

Live-code this:

```
python3 --version  
pip3 --version
```

Line-by-line explanation:

- `python3 --version` — asks the Python interpreter installed on this machine to report its own version. The explicit 3 matters: on many systems, plain `python` (no 3) either doesn’t exist, or points at an old Python 2 installation — a genuinely common and confusing trap, worth naming explicitly even though this course standardizes on `python3` everywhere specifically to sidestep it.
- `pip3 --version` — `pip` is Python’s package installer; confirming it works now means it won’t surprise you with a failure the first time the course needs it.

Run it. Expected output (versions will vary by machine):

```
Python 3.11.4  
pip 23.2.1 from /usr/local/lib/python3.11/site-packages/pip (python 3.11)
```

Common student mistakes to watch for:

- command not found: `python3` — almost always a PATH problem from an installation that didn’t add Python to the system PATH, or (on Mac) a case where only the Xcode Command Line Tools’ minimal Python stub is present. Do not debug this from scratch live; route to the precourse setup page and flag for follow-up, per the setup checklist above.
- A version below 3.10 reported successfully — the command *works*, so this can slip past a quick glance; explicitly tell the room to actually read the number, not just confirm “something printed.”
- Confusing `pip3` (the package manager) with `python3` (the interpreter) as interchangeable — they’re two different programs installed together; a 10-second clarification here prevents confusion the first time the course uses `pip3 install`.

Check for understanding: “If your `python3 --version` shows 3.9.1, is that a problem, and why might a specific version threshold matter for a class?” (Yes, borderline — some f-string format-spec behavior and syntax used later in the course assumes 3.10+; consistency across the whole room’s environment also avoids “it works on my machine” debugging later.)

6.4 Exercise 3 — Write `hello.py` (0:18–0:26, 8 min)

Teaching goal: The complete save-and-run loop — write code in an editor, save it, run it from the terminal — that every remaining lab this semester depends on.

Say to the class:

“One line of code, but the process is the point: write it in VS Code, save it, then go back to the terminal and run it. This loop — edit, save, run, read the output — is what you’ll do hundreds of times this semester.”

Do, live:

1. In VS Code, with the `ism2411` folder open, create a new file named exactly `hello.py`.
2. Type:

```
print("Hello, ISM2411!")
```

3. Save (Cmd+S / Ctrl+S) — narrate this explicitly: “If you don’t save, the terminal runs whatever was last saved to disk, not what’s currently on your screen. This is the single most common ‘my code isn’t working’ complaint that turns out to be an unsaved file.”
4. In the terminal, confirm you’re in the same folder as `hello.py` (`pwd`, `ls` to see the file listed), then run:

```
python3 hello.py
```

Line-by-line explanation:

- `print("Hello, ISM2411!")` — `print()` is a built-in function; whatever’s inside the parentheses (here, a string in quotes) gets written to the terminal. This is the first time this semester students are told explicitly: `print()` is *how* a Python script produces visible output — a script with no `print()` calls can run successfully and show nothing at all, which surprises beginners.
- `python3 hello.py` — this is the command that starts the interpreter and hands it your file to execute, top to bottom.

Run it. Expected output:

```
Hello, ISM2411!
```

Common student mistakes to watch for:

- Running `python3 hello.py` from the *wrong* directory (parent of `ism2411`, or from inside VS Code’s terminal which opened somewhere unexpected) — produces `can't open file 'hello.py': [Errno 2] No such file or directory`. This is deliberately *not* fixed here — it’s the exact scenario Exercise 7 explores in depth, so if it happens now, say so explicitly (“hold that thought, we’re going to explore exactly this in twenty minutes”) rather than fully

resolving it.

- Typing `Print` (capital P) — Python is case-sensitive, and `Print` is not a recognized name, producing a `NameError`. Good, low-stakes error to show live if it doesn't happen naturally in the room.
- Forgetting the closing parenthesis or a quote mark — produces a `SyntaxError`, which is a preview of Exercise 6's whole point; if this happens organically here, it's a good early, low-pressure look at what a `SyntaxError` looks like before Exercise 6 makes it the deliberate focus.

Check for understanding: “If I edit this file right now, changing the text inside the quotes, but *don't* save — what happens when I run `python3 hello.py` again?” (Nothing changes — the terminal runs whatever's actually saved on disk, which is the old version. Have someone demo this live if there's time; it's a genuinely sticky lesson.)

6.5 Exercise 4 — Path Practice (0:26–0:32, 6 min)

Teaching goal: Reinforce that `cd` moves *relative to where you currently are*, and that `pwd` is the reliable way to confirm your location rather than guessing.

Say to the class:

“Practice moving up and back down without looking at a file browser at all — just `cd` and `pwd`, back and forth.”

Live-code this:

```
pwd
cd ..
pwd
cd ism2411
pwd
```

Line-by-line explanation:

- `cd ..` — `..` is a special name meaning “the parent of the current directory,” identical to the `..` seen in Module 01's relative-path exercises. This moves you *up* one level.
- `cd ism2411` — moves back down into `ism2411`, assuming you're currently in its parent (which you are, right after the `cd ..` above) — this only works because `ism2411` exists as a subfolder *of wherever you currently are*; it is not a global shortcut to the `ism2411` folder from anywhere on the machine.

Run it. Expected output: `pwd` alternates between the `ism2411` path and its parent path, confirming each move.

Common student mistakes to watch for:

- Running `cd ism2411` from somewhere that is *not* its direct parent (e.g. after a second stray `cd ..`) — produces No such file or directory, since `cd` without a leading `/` looks for a subfolder of the *current* location, not a global search. This is worth explicitly contrasting with Module 01's absolute vs. relative path vocabulary: `ism2411` here is a relative reference, and relative references only work from the right starting point.

Check for understanding: “If I’m two levels above `ism2411` right now, what single command gets me back in — one `cd`, or do I need more than one?” (Depends on the exact starting point — the point of the question is to get students reasoning about relative position rather than memorizing a fixed command; a reasonable answer names either `cd ism2411` from one level up, or a longer relative/absolute path from further away.)

6.6 Exercise 5 — Make a Sub-Project (0:32–0:40, 8 min)

Teaching goal: Combine everything so far — `mkdir`, `cd`, writing a file, running it — into one continuous sequence, with a script that’s personalized rather than copy-pasted verbatim.

Say to the class:

“Same loop as Exercise 3, but nested one level deeper, and this time the script is about you specifically — your name and your major.”

Live-code this (using your own name/major as the example):

```
mkdir module02
cd module02
```

Then in VS Code, create `module02/hello.py`:

```
print("Alex Chen – Finance major")
```

Then in the terminal:

```
python3 hello.py
```

Line-by-line explanation: Nothing new syntactically from Exercise 3 — the teaching value here is entirely in the *directory structure*: point out explicitly that there are now **two different files both named `hello.py`**, living in two different directories (`ism2411/hello.py` and `ism2411/module02/hello.py`), and that this is completely fine, because — callback to Module 01’s Appendix B extra — filenames only need to be unique *within* a directory, not across the whole filesystem.

Run it. Expected output (with your own name/major substituted):

```
Alex Chen – Finance major
```

Common student mistakes to watch for:

- Editing the *original* `hello.py` from Exercise 3 instead of creating a new file inside `module02/` — check this by having students run `pwd` right before they run `python3 hello.py`, confirming they’re inside `module02`, not the parent.
- Using an em dash (–) vs. a plain hyphen (-) inconsistently when typing their name/major line — purely cosmetic, not worth correcting unless a student asks, but worth having a consistent example on your own screen.

Check for understanding: “Both `hello.py` files print something different when I run `python3 hello.py` — how does Python know which one to run each time?” (It runs whichever `hello.py` exists in the *current working directory* at the moment you run the command — there’s no ambiguity because you’re never running both at once; `cd` determines which one `python3 hello.py` finds.)

6.7 Exercise 6 — Deliberate Error Experiment (0:40–0:50, 10 min)

Teaching goal: The most important exercise in this lab. Reading a traceback carefully — line number, error type, and message — is a skill every single future lab depends on, and this is the first dedicated practice at it.

Say to the class:

“You’re about to deliberately break your own code and read the error message on purpose. This is not a distraction from real programming — reading error messages is real programming. Every professional developer spends a large fraction of their time doing exactly this.”

Do, live, using the Exercise 3 `hello.py`: Introduce a typo — change `print` to `prmt`:

```
prmt("Hello, ISM2411!")
```

Run it (`python3 hello.py`). It produces:

```
Traceback (most recent call last):
  File "hello.py", line 1, in <module>
    prmt("Hello, ISM2411!")
    ^^^^^
```

NameError: name 'prmt' is not defined. Did you mean: 'print'?

Walk the traceback from the bottom up — this order matters and is worth stating explicitly, since beginners often read top to bottom and get lost in the “Traceback (most recent call last)” preamble first:

- **Bottom line, the error itself:** NameError: name 'prmt' is not defined — the error *type* is NameError, and the message explains exactly what’s wrong: Python looked for something named `prmt` (expecting it to be a function, since it’s followed by parentheses) and found nothing by that name. Recent Python versions (3.10+) go a step further and append a helpful guess — Did you mean: 'print'? — point this out explicitly; it’s a real, useful feature, not a fluke, and students should get in the habit of reading all the way to the end of the error message rather than stopping at the first clause.
- **The line just above it, with the `^^^^` marker:** shows *exactly* which part of the line Python was confused by — here, the word `prmt` itself.
- **File "hello.py", line 1, in <module>:** tells you *where* — this file, this line number. For a one-line script this is trivial, but in a 40-line script, this line number is the fastest way to find the problem.

Now try the second type of error — remove a closing parenthesis:

```
print("Hello, ISM2411!"
```

Run it. It produces:

```
File "hello.py", line 1
  print("Hello, ISM2411!")
    ^
```

SyntaxError: '(' was never closed

Explain the difference between the two error types explicitly, since this is the exercise's actual required written deliverable:

"NameError means Python understood the structure of your code just fine, but couldn't find something you referred to by name — like calling someone's name in a room and nobody answering. SyntaxError means Python couldn't even understand the grammar of what you wrote — like an incomplete sentence. Different category of problem, different place to look for the fix: a NameError means check spelling and definitions; a SyntaxError means check punctuation — matching quotes, parentheses, colons."

Have every student do this analysis themselves, writing down for each of the two errors: (a) the line number reported, (b) the error type, (c) one sentence on what the message tells them about the problem — this is the graded deliverable for this exercise, not just watching your demo.

Then have them fix both and confirm hello.py runs cleanly again — do not skip this step; ending on a broken file undermines the exercise's confidence-building purpose.

Common student mistakes to watch for:

- Panicking at the wall of text and not reading past the first line ("Traceback (most recent call last):") — explicitly tell the room that line is boilerplate, always the same, and never contains the actual problem; the useful information is at the *bottom*.
- Trying to fix the error by guessing/randomly changing code rather than reading the message — this is exactly what reflection question 2 (tonight's homework) asks students to self-assess honestly, so naming the temptation explicitly now, in class, primes a more honest answer later.

Check for understanding: "Without running it — if I write `pint("hi")` instead of `print("hi")`, what error type and roughly what message do you expect?" (NameError: name 'pint' is not defined — the same category as the `print` typo, since it's still a misspelled name being called as a function.)

6.8 Exercise 7 — Run from the Wrong Directory (0:50–0:58, 8 min)

Teaching goal: Directly connect “current working directory” to “why my script suddenly can’t be found” — resolving the confusion that may have already surfaced organically back in Exercise 3.

Say to the class:

“One more directory-awareness exercise, and then we’re done with pure terminal mechanics for today. This one explains a failure mode you’ll hit constantly if you don’t understand it now.”

Do, live, starting from inside module02/:

```
cd ..  
python3 hello.py
```

This fails with:

```
python3: can't open file '/Users/yourname/ism2411/hello.py': [Errno 2] No such file or directory
```

Explain why, explicitly: `python3 hello.py` looks for `hello.py` in the *current* working directory — which, after `cd ..`, is `ism2411/`, not `ism2411/module02/`. Wait — actually there IS a `hello.py` directly in `ism2411/` too, from Exercise 3! So depending on which `hello.py` a student is thinking of, this might *not* fail — it might just run the *wrong* one (Exercise 3’s “Hello, ISM2411!” instead of Exercise 5’s personalized line). **This is worth calling out explicitly as an even sneakier failure mode than a clean error:** ask the room, “did anyone get output instead of an error just now — and if so, was it the output you expected?” If some students got Exercise 3’s output instead of an error, use that to make the point even more sharply: **a wrong-directory mistake doesn’t always announce itself with an error at all — sometimes it just silently runs the wrong file.**

Now run it correctly, using a relative path to reach into the subfolder:

```
python3 module02/hello.py
```

This succeeds, printing the Exercise 5 output:

```
Alex Chen – Finance major
```

Line-by-line explanation:

- `python3 module02/hello.py` — this argument is itself a *relative path*, exactly like Module 01’s Exercise 5 relative paths. `python3` doesn’t require you to `cd` into a folder to run a file inside it — you can instead point directly at the file’s path from wherever you currently are.

Common student mistakes to watch for:

- Concluding “the fix is always to `cd` into the right folder” and missing that a relative path works too, without moving at all — both are valid; make sure the room sees both options, since later labs frequently run a script from a fixed location while pointing at files elsewhere.
- Not noticing the silent-wrong-file case described above — walk the room specifically checking whose output was an error vs. whose was unexpectedly the *other* `hello.py`’s text.

Check for understanding: “Name two different ways to correctly run `module02/hello.py` while your terminal is sitting in `ism2411/.`” (Either `python3 module02/hello.py` directly, or `cd module02` followed by `python3 hello.py` — both work, for different reasons, and it’s worth having a student articulate why both work rather than just naming them.)

7 Wrap-Up (last ~7 minutes)

Review the reflection questions out loud:

1. *Has using only the terminal changed your mental model of where files live?* — No wrong answer; a strong response connects back to Module 01's path vocabulary — the terminal makes the directory hierarchy something you actively navigate, rather than something a file browser's icons abstract away.
2. *Exercise 6's error — instinct to read carefully, or start randomly changing things?* — Encourage total honesty here; this is genuinely diagnostic self-assessment, and the “randomly change things” instinct is extremely common and not something to be embarrassed about — the point is noticing it.
3. *Running reliably on a collaborator's machine with a different username/folder structure* — a strong answer notices that today's exercises used *absolute* assumptions in places (a specific folder name, a specific starting location) and starts to intuit why relative paths and portable project structure matter — this previews concerns that resurface when the course covers scripts that read/write files.

Review the submission checklist together:

- ☐ Screenshot shows Exercise 3's `hello.py` output (Hello, ISM2411!)
- ☐ Screenshot shows Exercise 5's `hello.py` output (name/major line)
- ☐ Screenshot includes `python3 --version` output
- ☐ Submitted to Canvas

Preview Module 03: “Today was almost entirely about the environment around your code — the terminal, the file system, running scripts. Starting next module, we focus entirely on the code itself: variables, types, and formatted output. The save-run-read loop you built today is the loop you'll run for the rest of the semester — it just gets more interesting content inside it.”

8 Appendix A — Full Command & Code Reference

Exercise 1:

```
pwd
ls
mkdir ism2411
cd ism2411
pwd
```

Exercise 2:

```
python3 --version
pip3 --version
```

Exercise 3 (ism2411/hello.py):

```
print("Hello, ISM2411!")
```

Exercise 4:

```
pwd
cd ..
pwd
cd ism2411
pwd
```

Exercise 5 (ism2411/module02/hello.py):

```
mkdir module02
cd module02
```

```
print("Alex Chen – Finance major")
```

Exercise 6 (two deliberate breaks, then fixed):

```
print("Hello, ISM2411!")      # NameError: name 'print' is not defined. Did you mean: 'print'
```

```
print("Hello, ISM2411!"      # SyntaxError: '(' was never closed
```

Exercise 7 (from ism2411/, one level above module02/):

```
python3 hello.py              # runs the WRONG hello.py (or errors, if Ex. 3's copy doesn't exist)
python3 module02/hello.py     # correct – runs Exercise 5's hello.py
```

9 Appendix B — Extra Practice (only if the class finishes early)

Seven required exercises fill the full 75 minutes at a normal pace, including realistic environment-troubleshooting slack. If a section moves unusually fast:

Extra — a third hello.py, three levels deep. Have students run `mkdir -p practice/deep/folder` (or three separate `mkdir + cd` steps if `-p` isn't covered) and write a third `hello.py` inside it. From `ism2411/`, have them run it with a single relative path (`python3 practice/deep/folder/hello.py`) without `cd`-ing there first — good extra rep of Exercise 7's relative-path lesson at greater depth.

Extra — one more deliberate error, a different type. Have students write `print("Hello" + 5)` and predict the error before running it. (`TypeError: can only concatenate str (not "int") to str` — a preview of exactly the type-mismatch error Module 03 explores in depth; a good bridge if you have a spare few minutes and want to plant a seed for next week.)